

Currency Module: Responses

1. Add Currency:

The screenshot shows a REST client interface for a project named 'Apricart Saudi'. The endpoint is `{{url}}/currencies` with a `POST` method. The request body is raw JSON:

```
{
  "active": true,
  "currencyCode": "EUR",
  "currencySymbol": "€",
  "currencyPrecision": "2",
  "currencyFormat": "1,00.00"
}
```

The response status is `201 Created` with a message `"Created"`. The response body is shown in JSON format:

```
{
  "status": 201,
  "message": "Created"
}
```

2. Get All Currencies

The screenshot shows a REST client interface for the same project. The endpoint is `{{url}}/currencies` with a `GET` method. The response status is `200 OK`. The response body is shown in JSON format:

```
{
  "data": [
    {
      "id": 1,
      "currencyCode": "USD",
      "currencySymbol": "$",
      "currencyPrecision": "2",
      "currencyFormat": "1,00.00",
      "active": true
    },
    {
      "id": 2,
      "currencyCode": "INR",
      "currencySymbol": "Rupee",
      "currencyPrecision": "2",
      "currencyFormat": "1,00.00",
      "active": true
    },
    {
      "id": 3,
      "currencyCode": "PKR",
      "currencySymbol": "Rupee",
      "currencyPrecision": "2",
      "currencyFormat": "1,00.00",
      "active": false
    },
    {
      "id": 4,
      "currencyCode": "EUR",
      "currencySymbol": "€",
      "currencyPrecision": "2",
      "currencyFormat": "1,00.00",
      "active": true
    }
  ]
}
```

3. Get Active Currencies

The screenshot shows a REST client interface for the endpoint `GET {{url}}/currencies/active`. The response is a JSON object with the following structure:

```
1 {
2   "status": 200,
3   "message": "OK",
4   "data": [
5     {
6       "id": 1,
7       "currencyCode": "USD",
8       "currencySymbol": "$",
9       "currencyPrecision": 2,
10      "currencyFormat": "1,00.00",
11      "active": true
12    },
13    {
14      "id": 2,
15      "currencyCode": "INR",
16      "currencySymbol": "Rupee",
17      "currencyPrecision": 2,
18      "currencyFormat": "1,00.00",
19      "active": true
20    },
21    {
22      "id": 4,
23      "currencyCode": "EUR",
24      "currencySymbol": "€",
25      "currencyPrecision": 2,
26      "currencyFormat": "1,00.00",
27      "active": true
28    }
29  ]
30 }
```

The response status is 200 OK, with a time of 258 ms and a size of 827 B.

4. Get Currency By CurrencyID

The screenshot shows a REST client interface for the endpoint `GET {{url}}/currencies/3`. The response is a JSON object with the following structure:

```
1 {
2   "status": 200,
3   "message": "OK",
4   "data": {
5     "id": 3,
6     "currencyCode": "PKR",
7     "currencySymbol": "Rupee",
8     "currencyPrecision": 2,
9     "currencyFormat": "1,00.00",
10    "active": false
11  }
12 }
```

The response status is 200 OK, with a time of 275 ms and a size of 592 B.

5. Get Currency by CurrencyCode

The screenshot shows a REST client interface for a project named "Apricart Saudi". The selected endpoint is "Get Currency By Currency Code" with a GET method. The URL is `{{url}}/currencies/code/EUR`. The response status is 200 OK, with a time of 170 ms and a size of 589 B. The response body is displayed in JSON format:

```
1 {
2   "status": 200,
3   "message": "OK",
4   "data": {
5     "id": 4,
6     "currencyCode": "EUR",
7     "currencySymbol": "€",
8     "currencyPrecision": "2",
9     "currencyFormat": "1,00.00",
10    "active": true
11  }
12 }
```

6. Get Currency by CurrencySymbol

The screenshot shows a REST client interface for a project named "Apricart Saudi". The selected endpoint is "Get Currency By Currency Symbol" with a GET method. The URL is `{{url}}/currencies/symbol/$`. The response status is 200 OK, with a time of 199 ms and a size of 589 B. The response body is displayed in JSON format:

```
1 {
2   "status": 200,
3   "message": "OK",
4   "data": [
5     {
6       "id": 1,
7       "currencyCode": "USD",
8       "currencySymbol": "$",
9       "currencyPrecision": "2",
10      "currencyFormat": "1,00.00",
11      "active": true
12    }
13  ]
14 }
```

7. Get Currency Format by CurrencyID

The screenshot shows a REST client interface for the endpoint `GET {{url}}/currencies/format/1`. The response status is `200 OK` with a time of `254 ms` and a size of `481 B`. The response body is displayed in JSON format:

```
1 {
2   "status": 200,
3   "message": "OK",
4   "data": "1,00.00"
5 }
```

8. Get Currency Precision by CurrencyID

The screenshot shows a REST client interface for the endpoint `GET {{url}}/currencies/precision/1`. The response status is `200 OK` with a time of `250 ms` and a size of `475 B`. The response body is displayed in JSON format:

```
1 {
2   "status": 200,
3   "message": "OK",
4   "data": "2"
5 }
```

9. Format Currency value

The screenshot shows a REST client interface for the endpoint `GET {{url}}/currencies/format/10000/1`. The response status is `200 OK` with a time of `267 ms` and a size of `483 B`. The response body is displayed in JSON format:

```
1 {
2   "status": 200,
3   "message": "OK",
4   "data": "10,000.00"
5 }
```

10. Update Currency

The screenshot shows a REST client interface for a PUT request. The URL is `{{url}}/currencies`. The request body is a JSON object: `{ "id": 2, "currencyCode": "US" }`. The response status is 200 OK, with a time of 365 ms and a size of 590 B. The response body is a JSON object: `{ "status": 200, "message": "OK", "data": { "id": 2, "currencyCode": "US", "currencySymbol": "Rupee", "currencyPrecision": "2", "currencyFormat": "1,00.00", "active": true } }`.

```
PUT {{url}}/currencies
```

```
{
  "id": 2,
  "currencyCode": "US"
}
```

Status: 200 OK Time: 365 ms Size: 590 B Save as example

```
{
  "status": 200,
  "message": "OK",
  "data": {
    "id": 2,
    "currencyCode": "US",
    "currencySymbol": "Rupee",
    "currencyPrecision": "2",
    "currencyFormat": "1,00.00",
    "active": true
  }
}
```

11. Delete Currency

The screenshot shows a REST client interface for a DELETE request. The URL is `{{url}}/currencies/3`. The response status is 200 OK, with a time of 485 ms and a size of 503 B. The response body is a JSON object: `{ "status": 200, "message": "OK", "data": "Currency removed successfully" }`.

```
DELETE {{url}}/currencies/3
```

Status: 200 OK Time: 485 ms Size: 503 B Save as example

```
{
  "status": 200,
  "message": "OK",
  "data": "Currency removed successfully"
}
```